

Lab report -1:

In Java, multiple inheritance means a class can get features from more than one parent. An abstract class does not support multiple inheritance. A class can extend only one abstract class. An ~~inherit~~ interface supports multiple inheritance in Java. A class can implement more than one interface at the same time. Abstract classes can have both ~~abstract~~ and normal methods. Interfaces contain only abstract methods. We use an abstract class when classes are closely related. We use an interface when multiple inheritance is needed.

Therefore, interfaces are used to solve the multiple inheritance problem in Java.

2

Java code example:

```
interface A {  
    void show();  
}
```

```
interface B {  
    void print();  
}
```

```
class Test implements A, B {  
    public void show() {  
        System.out.println("Interface A");  
    }  
    public void print() {  
        System.out.println("Interface B");  
    }  
}
```

Lab report-2;

Encapsulation is an OOP concept that hides data and protect them from direct access. It ensures data security by making variable private inside a class. Outside classes can not change data directly. Data can be changed only using public methods. These methods check the values before saving them. This prevents wrong or harmful data from entering the system. Encapsulation also keeps data consistent and correct. If invalid data is given, the method rejects it. In bank system, this is very important for safety. Therefore, encapsulation ensures both data and security.

4

Java code:

```
class Account {  
    private String accountNumber;  
    private double balance;  
    public void setAccountNumber(String accNo) {  
        if (accNo != null && !accNo.isEmpty()) {  
            accountNumber = accNo;  
        }  
        else {  
            System.out.println("Invalid account number");  
        }  
    }  
    public void setInitialBalance(double amount) {  
        if (amount >= 0) {  
            balance = amount;  
        }  
        else {  
            System.out.println("Balance cannot be negative");  
        }  
    }  
}
```


Lab report - 4:

JDBC stands for Java Database connectivity. It is used to connect a Java program with a relational database. JDBC works as a bridge between Java application and database. The Java program sends SQL queries using JDBC. JDBC driver receives the request and talks to database. The database processes the query and sends the result back. JDBC driver receives and returns the result to the Java program. JDBC allows data to be inserted, updated, deleted and read. It also handles connection and error management. This JDBC manages smooth communication between Java and Database. To execute a SELECT query, JDBC follows some fixed steps. First, a connection is created with the database. Then a statement prepared statement object is created.

After that, a SELECT SQL query is written.

The query is executed using `executeQuery()` method. This method returns a `ResultSet` object. A `try` block is used to handle database operation safety. A `catch` block handles SQL runtime errors. Finally, a `finally` block is used to close connection and resources.

Java code:

```
import java.sql.*;
```

```
class selectExample {
```

```
    public static void main (String[] args) {
```

```
        Connection con = null;
```

```
        try {
```

```
            con = DriverManager.getConnection ("jdbc:mysql://localhost:3306/testdb", "root", "password");
```

```
            Statement st = con.createStatement();
```

```
            ResultSet rs = st.executeQuery ("SELECT * FROM student");
```



```
while(rs.next()) {  
    System.out.println(rs.getInt(1) + " " + rs.getString(2));  
  
}  
  
catch (Exception e) {  
    System.out.println("Error occurred");  
  
} finally {  
    try {  
        if (con != null)  
            con.close();  
    }  
    catch (Exception e) {  
        System.out.println("connection not closed");  
    }  
  
}
```

Lab report-5:

In a Java EE application, a servlet works as a controller. The controller manages the flow between model and view. The model contains business data and logic. The servlet gets data from the model, then it sends this data to view. JSP is used as the view to show data to the user. The servlet forwards the request to JSP using Request Dispatch. Data is sent using request attributes. JSP reads the data and displays it. Thus, the servlet controller controls the application flow.

Java code:

```
import java.io.*;
import javax.servlet.*;
import javax.http.*;
```



```

public class Helloservlet extends HttpServlet {
    protected void doGet(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        String name = "student";
        req.setAttribute("msg", name);
        RequestDispatcher rd = req.getRequestDispatcher(
            "Hello.jsp");
        rd.forward(req, res);
    }
}

```

JSP code for view:

```

<html>
<body>
    <h2> Hello, {msg} </h2>
</body>
</html>

```

Lab report-6:

Prepared statement is used to execute SQL queries safely in JDBC. It improves performance because the query is precompiled by the database. The same query can be used many times with different values. Prepared statement is faster than statement for repeated queries. It also improves security by preventing SQL injection attacks. User input is treated as data, not as SQL code. Statement directly executes SQL and is less secure. Prepared statement uses placeholders (?) for values. These values are set using setter methods. So prepared statement is better for performance and security.

11

Java code (Insert using PreparedStatement):

```
import java.sql.*;

class insert {
    public static void main (String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            String sql = "INSERT INTO student (id, name) values (?, ?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, 1);
            ps.setString(2, "Rahim");
            ps.executeUpdate();
            System.out.println("Record inserted");
            con.close();
        } catch (Exception e) {
            System.out.println("Error occurred");
        }
    }
}
```


Lab report-7:

ResultSet is an object in JDBC that stores data returned from a database query. It is mainly used with select queries.

ResultSet works like a table with rows and columns. The `next()` method moves the cursor to the next row. It returns true if data is available. The `getString()` method is used to read String type data, data is read column by column from ResultSet. ResultSet helps Java programs fetch database records easily. Thus it is very important for retrieving data in JDBC.

Java code (ResultSet usage):

```
import java.sql.*;
```

```
class ResultSetExample {
```

```
    public static void main (String[] args) {
```

```

Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb","root","password");

Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT id, name
    from student");

while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("Name");
    System.out.println("id: " + name);
}

con.close();
} catch (Exception e) {
    System.out.println("Error occurred");
}
}
}

```